

Verilog 期末设计实验报告

学生姓名	王一诺	学号	2312331	指导老师	董前琨老师
------	-----	----	---------	------	-------

一、实验选题

请使用本学期学到的 verilog 语法知识和 vivado 工具，设计并实现一个具有实际功能的电路模块，最好使用时序逻辑，可以采用如下备选设计或自行设计其他电路模块。

本次我选择设计一个交通信号灯控制系统，按照以下规则循环运行：

1. 红灯持续 30 秒，提示车辆停止。
2. 黄灯持续 3 秒，提示车辆准备行驶或停止。
3. 绿灯持续 25 秒，提示车辆通行。

二、设计思路

1. 状态机

使用 3 个状态表示交通灯颜色：RED（红灯）、YELLOW（黄灯）、GREEN（绿灯）

状态转换规则：红灯持续 30 秒后进入黄灯状态；黄灯持续 3 秒后进入绿灯状态；绿灯持续 25 秒后回到红灯状态。

状态控制：通过时钟驱动状态机，利用计数器控制每种状态的持续时间。

2. 计时器

每个状态都有一个计数器，用于记录该状态下的持续时间。计数器达到设定值时，状态机切换到下一个状态，并将计数器复位为 0。

计数范围：红灯计时：0 ~ 29，计数到 30 时切换到黄灯状态。黄灯计时：0 ~ 2，计数到 3 时切换到绿灯状态。绿灯计时：0 ~ 24，计数到 25 时切换到红灯状态。

3. 灯光输出控制

使用 3 个输出信号分别控制红灯（red）、黄灯（yellow）、绿灯（green）。根据当前状态，点亮对应灯光，其他灯光熄灭。

4. 工作流程

初始化：系统复位时，进入初始状态 RED（红灯），计数器清零。

运行：每秒钟时钟驱动计数器增加；计数器达到设定值时，状态切换并清零计数器；根据当前状态点亮对应的灯光。

循环切换：状态按照 RED -> YELLOW -> GREEN -> RED 的顺序循环，控制交通灯的正常运行。

三、设计代码

模块设计：

测试文件：

```

module traffic_light (
    input wire clk,           // 时钟信号, 假设频率为 1Hz
    input wire rst_n,         // 异步复位信号 (低电平有效)
    output reg red,           // 红灯输出
    output reg yellow,        // 黄灯输出
    output reg green          // 绿灯输出
);
// 状态定义
parameter RED = 2'b00;
parameter YELLOW = 2'b01;
parameter GREEN = 2'b10;
reg [1:0] state;            // 当前状态
reg [1:0] next_state;       // 下一个状态
reg [5:0] counter;          // 计数器, 用于时间控制
// 状态切换逻辑
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= RED;       // 异步复位到红灯状态
        counter <= 6'd0;     // 计数器清零
    end else begin
        state <= next_state; // 更新状态
        if (state == RED && counter == 6'd29) begin
            counter <= 6'd0;
        end else if (state == YELLOW && counter == 6'd2) begin
            counter <= 6'd0;
        end else if (state == GREEN && counter == 6'd24) begin
            counter <= 6'd0;
        end else begin
            counter <= counter + 1; // 时间计数
        end
    end
end
// 下一个状态生成逻辑
always @(*) begin
    case (state)
        RED: begin
            if (counter == 6'd29)
                next_state = YELLOW;
            else
                next_state = RED;
        end
        YELLOW: begin
            if (counter == 6'd2)
                next_state = GREEN;
            else
                next_state = YELLOW;
        end
        GREEN: begin
            if (counter == 6'd24)
                next_state = RED;
            else
                next_state = GREEN;
        end
        default:
            next_state = RED;
    endcase
end
// 输出控制逻辑
always @(*) begin
    case (state)
        RED: begin
            red = 1;
            yellow = 0;
            green = 0;
        end
        YELLOW: begin
            red = 0;
            yellow = 1;
            green = 0;
        end
        GREEN: begin
            red = 0;
            yellow = 0;
            green = 1;
        end
        default: begin
            red = 0;
            yellow = 0;
            green = 1;
        end
    endcase
end
endmodule

```

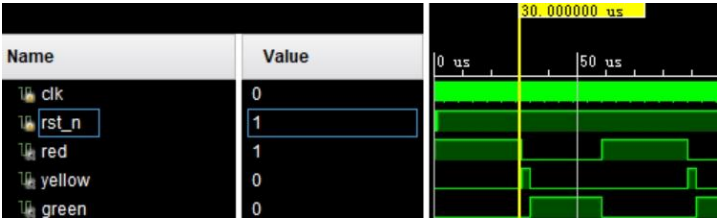
```

module traffic_light_tb;
// 信号定义
reg clk;
reg rst_n;
wire red, yellow, green;
// 实例化被测模块
traffic_light uut (
    .clk(clk),
    .rst_n(rst_n),
    .red(red),
    .yellow(yellow),
    .green(green)
);
// 时钟生成
initial begin
    clk = 0;
    forever #500 clk = ~clk; // 1Hz 时钟, 周期为 1000ns
end
// 测试过程
initial begin
    // 初始化
    rst_n = 0;
    #1000 rst_n = 1; // 1ms 后释放复位

    // 观察仿真 120 秒 (观察灯光完整切换过程)
    #120000000 $stop; // 停止仿真
end
endmodule

```

四、测试样例：



初始处于红灯状态，经过三十秒后，切换为黄灯。



黄灯持续 3s 后，进入绿灯状态。



在经过 25s 后，进入红灯状态。以此循环往复。